SMART TRAFFIC SIGNAL SIMULATOR

Subject: Foundations of Programming

Course Code:

Teacher name: Dr. Renuka Mane

Div: 3

Batch: C2

Team members: Om Jadhav – 1262252682

Niharika Singh – 1262253874

Yanika Phatak – 1262251867

Shriya Shelke - 1262252710

INTRODUCTION AND TECHNOLOGIES USED

INTRODUCTION	TECHNOLOGIES USED
Traffic congestion has become a major issue in urban areas due to the increasing number of vehicles on roads. Most traditional traffic signal systems operate on fixed time intervals, regardless of the actual traffic density at an intersection. This often leads to unnecessary waiting time, fuel wastage, and increased pollution.	▪ C Programming Language ▪ GCC / Turbo C Compiler ▪ Libraries: i) stdio.h ii) stdlib.h iii) string.h ▪ Console-based simulation

PROBLEM STATEMENT AND OBJECTIVES

What's the problem?	What's the objective?
Conventional traffic signal systems suffer from several limitations: ▪ Fixed signal timing doesn't consider real-time traffic conditions. ▪ Unnecessary long green signals for roads with fewer vehicles. ▪ Traffic congestion on roads. ▪ No special priority for emergency vehicles. ▪ Traffic imbalance leads to delays, fuel wastage, and frustration.	▪ To design and implement a smart traffic signal simulation using C programming. ▪ To dynamically allocate green signal time based on traffic density. ▪ To prioritize emergency vehicles by giving them immediate green signal. ▪ To simulate real-world traffic decision-making using simple logic.

ALGORITHM

Step 1: Start the program

Step 2: Initialize roads

Step 3: Begin infinite loop

Step 4: Take input

Step 5: Check for emergency vehicle

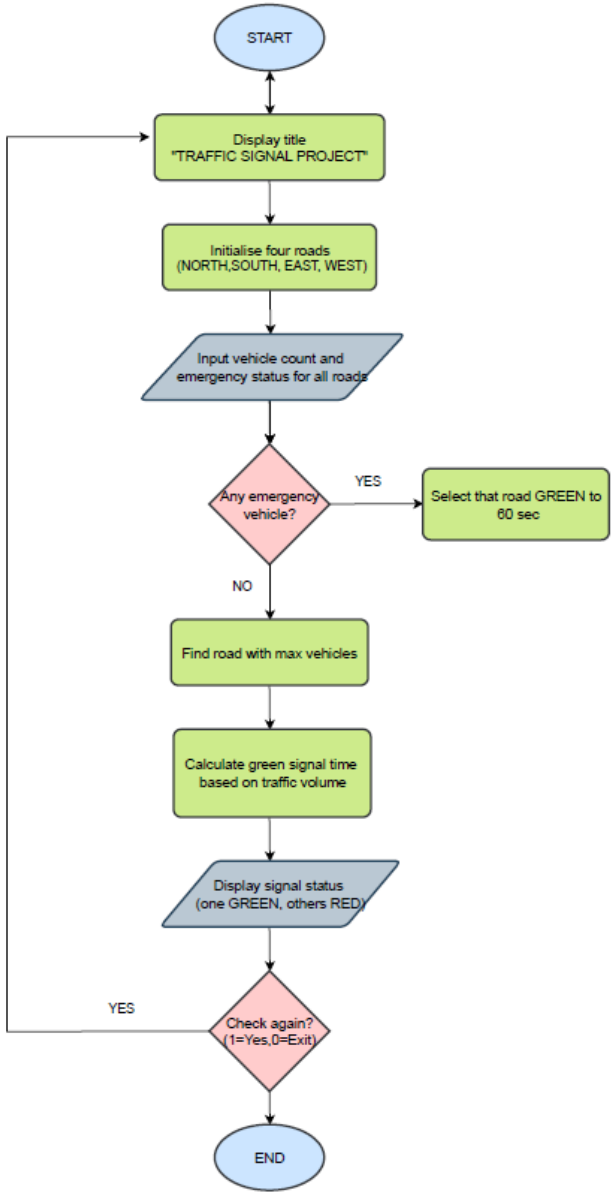
Step 6: Find maximum traffic

Step 7: Assign time based on traffic

Step 8: Display signal status

Step 9: Repeat or exit

FLOWCHART



STRUCTURE USED

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

//structure to store road details
struct Road {
    char name[20];
    int count;       // number of cars
    int emergency;   // 1 means yes ambulance, 0 means no
};

```

We include **stdio.h** for printing, **stdlib.h** to clear the screen, and **string.h** to handle the road names.

struct Road acts as a container to store road data.
name → Road name
count → Number of cars
emergency → 1 if ambulance present, else 0

MAIN FUNCTION & ROAD INITIALIZATION

```c
// defining functions
void input(struct Road r[]);
int findGreenRoad(struct Road r[]);
int getTime(int count);
void showSignal(struct Road r[], int greenIndex, int time);

int main() {
    struct Road r[4];
    int choice;

    //names of the roads
    strcpy(r[0].name, "NORTH");
    strcpy(r[1].name, "EAST");
    strcpy(r[2].name, "SOUTH");
    strcpy(r[3].name, "WEST");

    while(1) {
        // clear the screen (cls is for windows)
        system("cls");

        printf("\n TRAFFIC SIGNAL PROJECT \n");

        // 1. calling input function
        input(r);

        // 2. logic to find which road will be green
        int greenIndex = findGreenRoad(r);

        // 3. getting the time duration
        int time = getTime(r[greenIndex].count);

        // if ambulance is there, full 60 seconds directly
        if(r[greenIndex].emergency == 1) {
            time = 60;
        }

        // 4. print the final signal table
        showSignal(r, greenIndex, time);
```

```c
        printf("\nCheck again? (1 = Yes, 0 = Exit): ");
        scanf("%d", &choice);

        // if user types 0 then loop stops
        if(choice == 0) {
            break;
        }
    }

    return 0;
}
```

Here, an array of 4 roads is created, i.e NORTH, EAST, SOUTH, WEST.

Names are assigned using strcpy(). This is because we can't use = for words in C.

while(1) creates a continuous running system. (infinite loop)

TAKING INPUT FOR VEHICLE COUNT

```c
// function to take user input for all 4 roads
void input(struct Road r[]) {
    int i;
    // loop running 4 times for each road
    for(i = 0; i < 4; i++) {
        printf("\n %s Road \n", r[i].name);
        printf("Vehicles: ");
        scanf("%d", &r[i].count);

        // if user enters negative number make it 0
        if(r[i].count < 0) r[i].count = 0;

        printf("Emergency (1=Yes, 0=No): ");
        scanf("%d", &r[i].emergency);
    }
}
```

A for loop is used here. (runs 4 times).

We take the car count and emergency status.

Incase a user enters a negative value, it gets corrected to 0. This ensures valid data input.

GREEN SIGNAL LOGIC (CORE LOGIC)

```c
// logic to find road with max traffic
int findGreenRoad(struct Road r[]) {
    int i;
    int maxIndex = 0;
    int maxVal = -1;

    // checking if any emergency vehicle is there first
    for(i = 0; i < 4; i++) {
        if(r[i].emergency == 1) {
            return i; // if yes then return this road index
        }
    }

    // if no emergency check normal traffic count
    for(i = 0; i < 4; i++) {
        if(r[i].count > maxVal) {
            maxVal = r[i].count;
            maxIndex = i;
        }
    }
    return maxIndex;
}
```

This is the smartest part of the code.

Step 1: We check emergency first. If an emergency vehicle is present on any road, it ignores the vehicle count on other roads.

Step 2: If there is no emergency, the road with maximum number of cars is chosen to give the green light to.

TIME ALLOCATION LOGIC

```
// loop to decide time based on cars
int getTime(int count) {
    if(count > 20) {
        return 60; // heavy traffic
    } else if(count >= 10) {
        return 40; // medium traffic
    } else {
        return 20; // low traffic
    }
}
```

This is a simple logic to decide the green light duration.

Heavy Traffic (>20 cars): 60 seconds
Medium Traffic (10-20 cars): 40 seconds
Light Traffic (<10 cars): 20 seconds

THE DISPLAY

```c
// the output table
void showSignal(struct Road r[], int greenIndex, int time) {
    int i;
    printf("       SIGNAL STATUS        \n");

    // warning for emergency
    if(r[greenIndex].emergency == 1) {
        printf(" !! EMERGENCY VEHICLE !! \n");
    }

    // green for one road and red for others
    for(i = 0; i < 4; i++) {
        if(i == greenIndex) {
            printf(" %s: \tGREEN (%d sec) \t[Cars: %d]\n", r[i].name, time, r[i].count);
        } else {
            printf(" %s: \tRED \t\t[Cars: %d]\n", r[i].name, r[i].count);
        }
    }
}
```

Here, we loop through the 4 roads.

The selected road prints **GREEN** along with the time.
The other roads print **RED**.

Then we display the formatted output table.

CONCLUSION & FUTURE SCOPE

CONCLUSION	FUTURE SCOPE
In this project, we successfully developed a Smart Traffic Signal System simulation using C programming. The system intelligently determines which road should receive the green signal based on two main factors: emergency priority and traffic density.	▪ Integration with real-time traffic sensors. ▪ AI-based prediction of traffic patterns. ▪ Integration with smart city infrastructure. ▪ Adding pedestrian crossing management. ▪ Using cameras for vehicle detection instead of manual input.

THANK YOU